# 疎行列における実部と虚部の分離と棄却による BiCG 法の高速化

# ① 研究概要と既存手法

② 提案手法（BC 棄却, BCCSR 形式）

③ 性能評価

④ まとめ

# 概要

## 背景

- $A\mathbf{x} = \mathbf{b}$ を解く ILUBiCG 法はメモリ使用量が多く, 1 反復毎の処理量が多い
- 前処理行列における要素の棄却により演算量は削減できるが, 収束性は悪化

## 目的

- 棄却による収束性の悪化を抑制
- メモリ使用量, 計算時間の削減

## 手法

- 行列の実部と虚部を分けて格納（BCCSR 形式）・棄却（BC 棄却）

## 結果

- SpMV 最大 3.5x, SpTRSV 最大 2.0x, $A$ のメモリを最大 38% 削減 (CSR比)
- 様々な問題で BC 棄却が収束性悪化を抑制（従来の棄却比）
- ABMC 法と BCCSR 形式の相性◎（$P$ のメモリ削減, SpTRSV の実行速度）

# 研究対象

- 複素線型方程式 $A\mathbf{x} = \mathbf{b}$
  - $A$: 疎行列. サイズ $n \times n$, 非ゼロ要素 $m$ 個
- 有限要素法によるシミュレーションで現れる
  - 電磁場解析, 音響解析, 地震波動解析, etc…
- シミュレーションのボトルネックになりがち
- $A$ は大規模で疎なので, 直接法ではなく反復法で解く
  - 解が収束するまで同じ操作を反復

| | 空間計算量 | 時間計算量 |
|---|---|---|
| 直接法 | $O(n^2)$ 😖 | $O(n^3)$ 😖 |
| 反復法（CG 法） | $O(m)$ 😀 | 理論的には $O(nm)$ 😀 |

# ILUBiCG 法

- BiCG 法
  - 複素線型方程式 $A\mathbf{x} = \mathbf{b}$ を解く反復法の一種
  - $A$ が単位行列に近いほど収束性が良い（少ない反復回数で収束）
  - 反復計算部は以下の操作で構成
    - AXPY（ベクトルの加算・定数倍），内積
    - SpMV（疎行列ベクトル積）
  - 一般に，行列は CSR 形式を用いて格納

疎行列⇔CSR 形式

| | | | | |
|---|---|---|---|---|
| 1 | | | | |
| | 5 | | | |
| 2 | | -1 | | |
| | | 2 | 3 | |

⇔

| | | | | | | |
|---|---|---|---|---|---|---|
| *val* | 1 | 5 | 2 | -1 | 2 | 3 |
| *col_idx* | 0 | 1 | 0 | 2 | 2 | 3 |
| *row_ptr* | 0 | 1 | 2 | 4 | 6 | |

- ILUBiCG 法
  - ILU 分解による前処理を加えた BiCG 法．収束性向上
  - 反復計算部に SpTRSV（前進後退代入）が追加
    （$L\mathbf{x} = \mathbf{b}$ / $U\mathbf{x} = \mathbf{b}$ を解く．$L$：下三角，$U$：上三角）
- 本研究では代表的な反復法である ILUBiCG 法を高速化・省メモリ化する

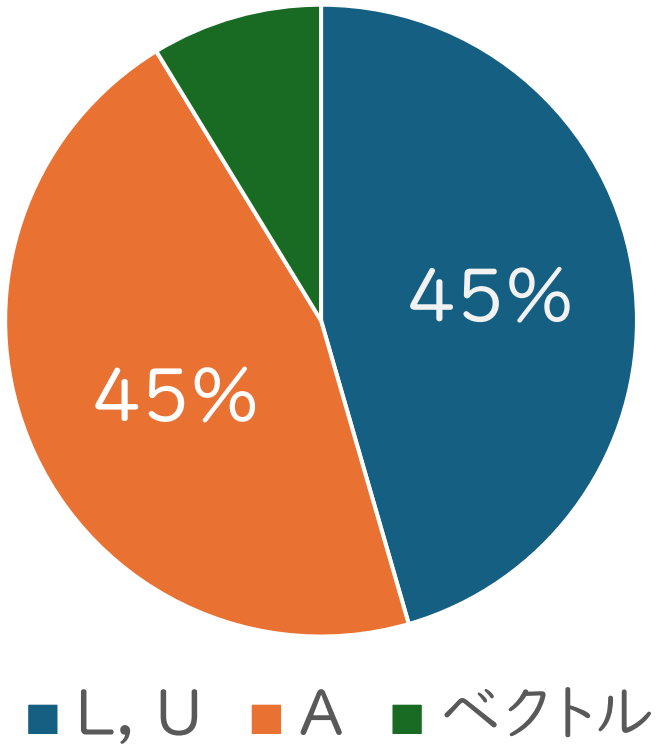
# 反復法における前処理

- ILU 分解によって前処理行列 $L, U$ を得る（$LU \approx A$）
- $A\mathbf{x} = \mathbf{b}$ の代わりに $(LU)^{-1}A\mathbf{x} = (LU)^{-1}\mathbf{b}$ を解く
- 収束性を改善させる役割
  - $(LU)^{-1}A$ は単位行列に近くなるため，良条件に
  - $A \approx LU$ の近似精度が高いほど収束性が向上

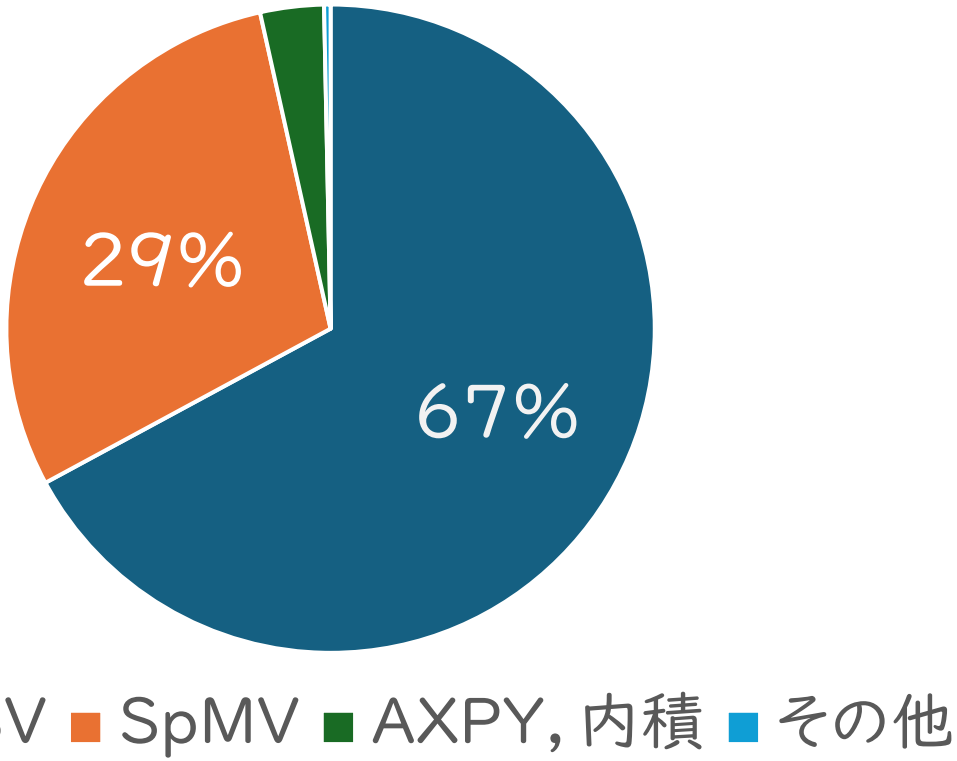
# ILUBiCG 法の課題

- 係数行列 $A$ と前処理行列 $L, U$ を保持するため, メモリ使用量が多い
- 反復毎の演算量が多く, SpTRSV と SpMV がボトルネック

メモリ使用量（行列DF2, CSR 形式）




実行時間（行列DF2）



# 前処理行列における棄却

- ポストフィルタリング
  - ILU 分解で得た前処理行列のうち, ノルムが微小な要素を棄却する
  - 非ゼロ要素が減ることで, メモリ使用量と反復毎の演算量を削減😀
  - $A \approx LU$ の近似精度が悪化し, 収束性が低下 😖
- 最も典型的な手法では, ユーザが与える閾値 $t$ を用いて, 各要素のノルムで判定

$$L_{ij} \leftarrow 0 \quad \text{if } |L_{ij}| < t \ (i \neq j)$$

行列要素の棄却の図

| | | | |
|---|---|---|---|
| 8+5i | | | |
| 4+i | 4-8i | | |
| | 3+2i | 4+8i | |
| 1+2i | | 1-i | 10 |

$t=2.5$ ⇨

| | | | |
|---|---|---|---|
| 8+5i | | | |
| 4+i | 4-8i | | |
| | 3+2i | 4+8i | |
| | | | 10 |

# 研究の目的と方針

## 目的

- メモリ使用量・実行時間を削減する
- ポストフィルタリングでの収束性悪化を抑制したい

## 方針

- 応用で現れる複素線型方程式における典型的な特徴を利用
  1. 非ゼロ要素の実部と虚部の絶対値の大きさに差がある
     ⇒ 値を変更する量を少なくするポストフィルタリング（BC 棄却）を設計
  2. 非ゼロ要素の個数が実部と虚部で偏っている
     ⇒ 複素行列向けの格納形式（BCCSR 形式）を設計

# 関連研究（ポストフィルタリング）

- 棄却の基準についての既存研究
  [Y. Saad. ILUT: A dual threshold incomplete LU factorization. 1994.]
  [E. Chow. A Priori Sparsity Patterns for Parallel Sparse Approximate Inverse Preconditioners. 2000.]
  [M. L. Barton. Three-dimensional magnetic field computation on a distributed memory parallel processor. 1990.]
  [I. E. Kaporin and I. N. Kon'shin. Post-filtering of IC2-factors for load balancing in parallel preconditioning. 2009.]
  - 非ゼロ要素の絶対値の大きさ
  - 非ゼロ要素の位置
  - 各行における非ゼロ要素数

いずれも実/複素を区別しない条件

- 複素数に特化した棄却手法が上木らによって提案された
  [上木脩生, 桝井晃基, 伊野文彦. 前処理行列の虚部切り捨てによる複素対称線形方程式向け反復法の高速化. 2024.]
  - 実部と虚部の収束性に対する影響の非対称性を指摘
  - 虚部を全て棄却する手法
- 本研究は上木らの研究の拡張として位置づけられる

① 研究概要と既存手法

# ② 提案手法（BC 棄却，BCCSR 形式）

③ 性能評価

④ まとめ

# 方針1について

- 特徴1：非ゼロ要素の実部と虚部の絶対値の大きさに差がある
- 従来の棄却では，必ず実部と虚部をどちらも棄却する
- 絶対値の小さい実部または虚部の一方のみを棄却できれば，前処理行列の数値的な変更を小さくできそう

# 【提案手法 I】BC(bipartite complex) 棄却

- ユーザが与える閾値 $t'$ を用いて，実部と虚部を独立に棄却
- 実部と虚部の一方が小さい要素も棄却できるようになる
- BCCSR 形式（後述）と併用

$$\mathrm{Re}[L_{ij}] \leftarrow 0 \quad \text{if } |\mathrm{Re}[L_{ij}]| < t' \ (i \neq j)$$
$$\mathrm{Im}[L_{ij}] \leftarrow 0 \quad \text{if } |\mathrm{Im}[L_{ij}]| < t' (i \neq j)$$

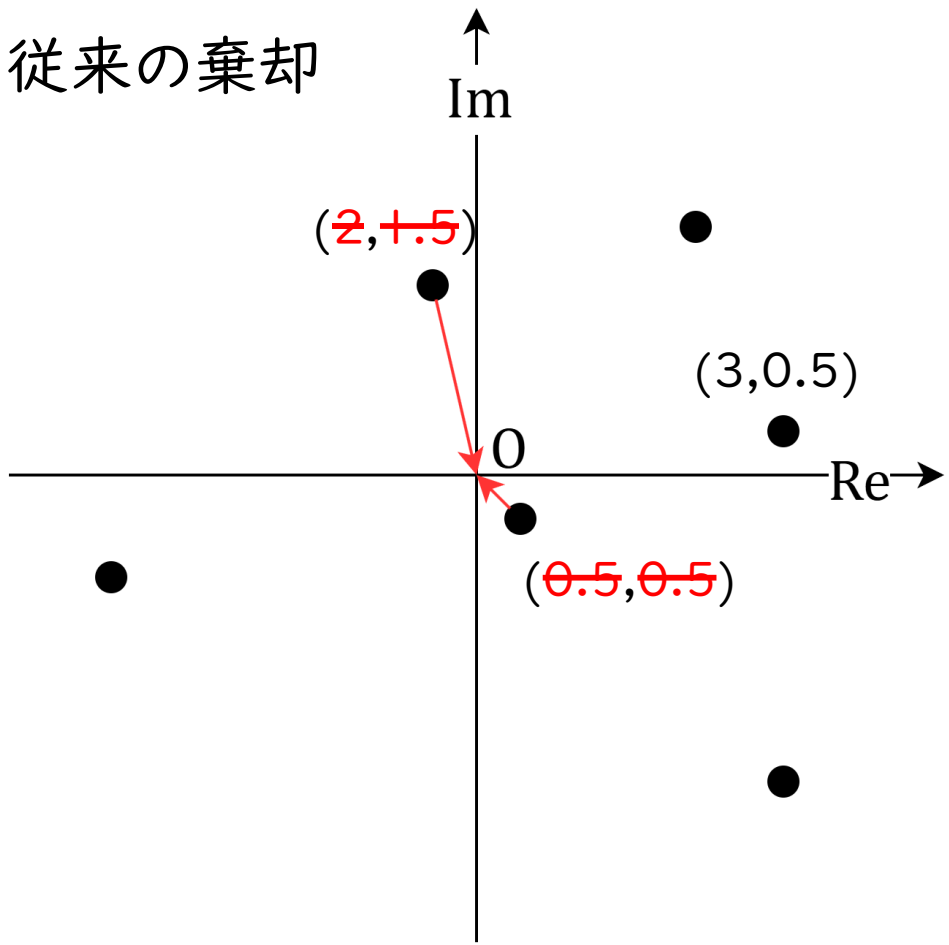
| | | | |
|---|---|---|---|
| 8+5i | | | |
| 4+i | 4-8i | | |
| | 3+2i | 4+8i | |
| 1+2i | | 1-i | 10 |

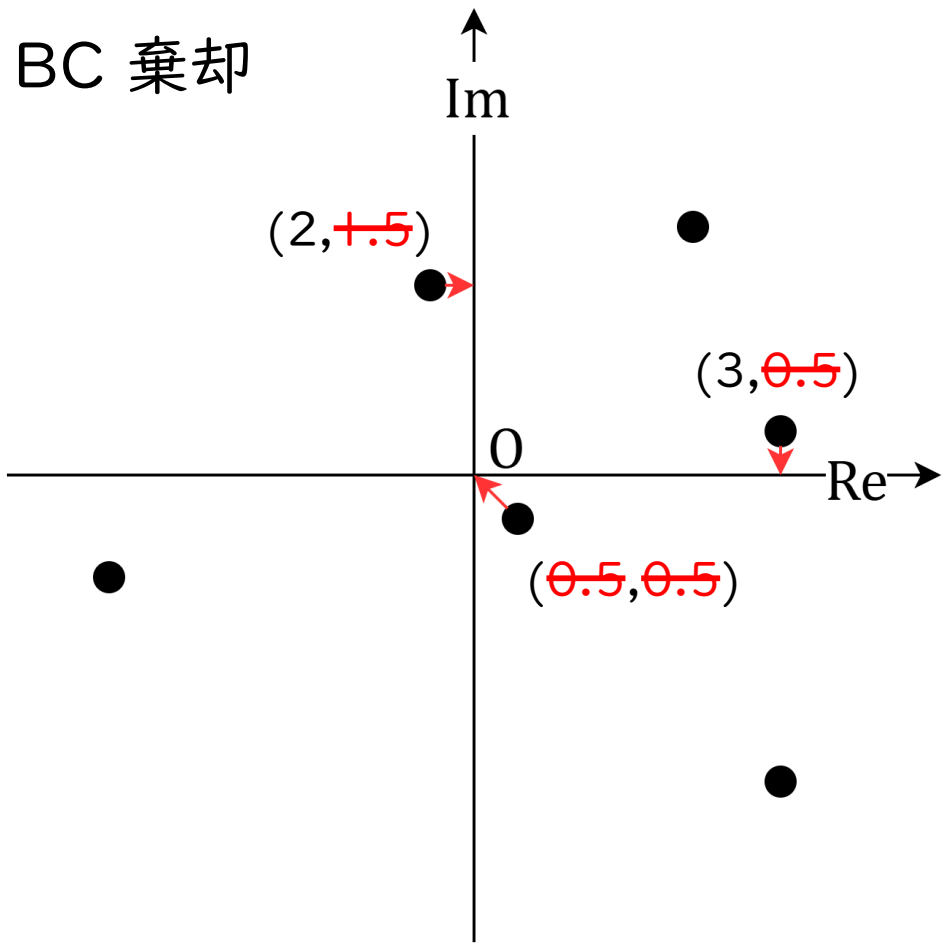
$t'=1.5$ ⇨

| | | | |
|---|---|---|---|
| 8+5i | | | |
| 4 | 4-8i | | |
| | 3+2i | 4+8i | |
| 2i | | | 10 |

# 複素平面における棄却の可視化

- 従来の棄却：原点に近い要素を原点に移動
- BC 棄却：軸に近い要素を軸に移動



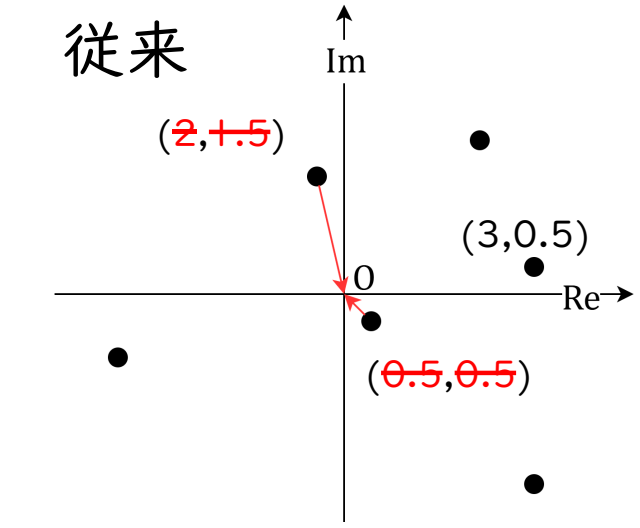



# BC 棄却の性質

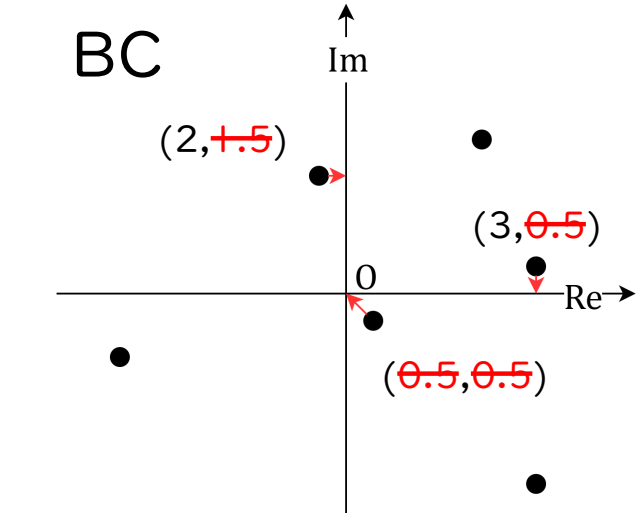
- 各点の移動距離（赤矢印）が大きいほど，前処理行列の近似精度が損なわれる
- BC 棄却はこの点で従来の棄却よりも理論的に優れる

定理：同じ個数の実数成分を棄却するとき，
BC 棄却は，従来の棄却よりも，
各点の移動距離の二乗和を小さくできる

- 棄却による $L, U$ の変更差分が小さいので，収束性への影響も小さいだろうという期待 🥲 💭




$(2^2 + 1.5^2) + (0.5^2 + 0.5^2) = 6.75$




$1.5^2 + 0.5^2 + (0.5^2 + 0.5^2) = 3$

# 方針2について

- 特徴2：非ゼロ要素の個数が実部と虚部で偏っている
- std::complex（C++）などの実部と虚部の組を1要素として扱う格納形式では，実数または純虚数に含まれるゼロは削減できない

| | | | |
|---|---|---|---|
| 1+i | | | |
| | 5 | | |
| 2 | | -i | |
| | | 1+3i | 2+4i |

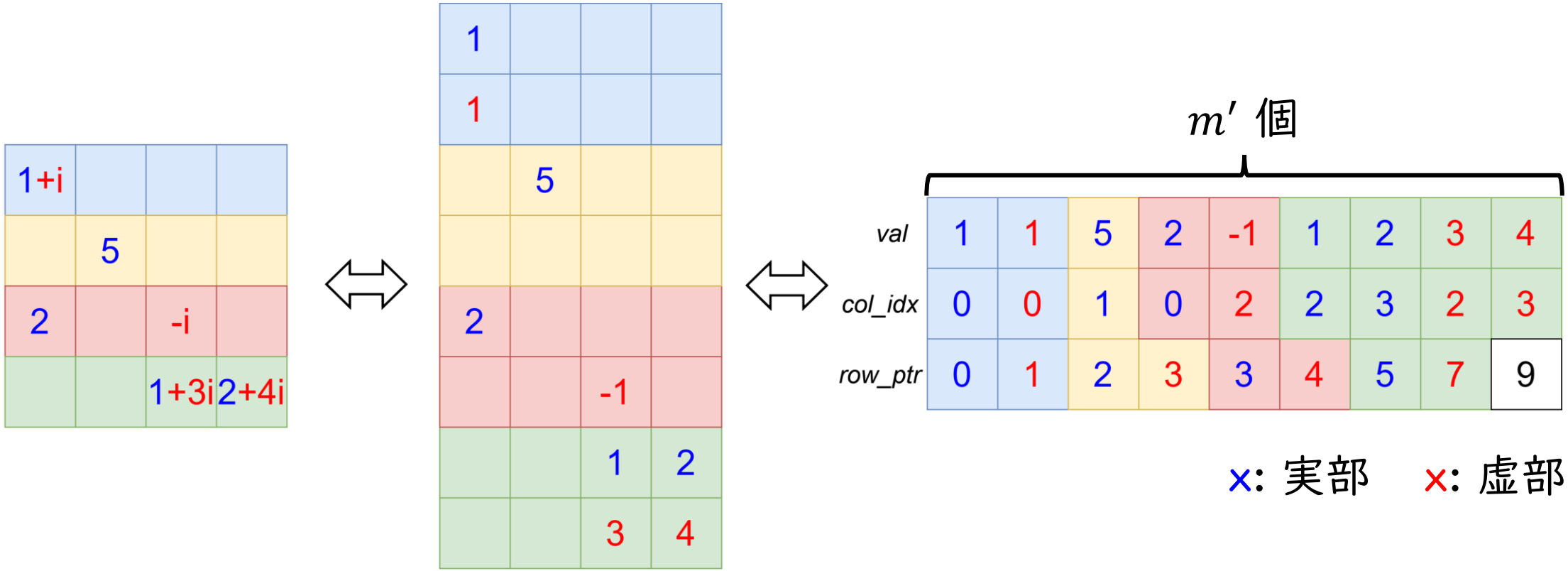
⇔

$m$ 個

| | | | | | | |
|---|---|---|---|---|---|---|
| *val* | (1,1) | (5,0) | (2,0) | (0,-1) | (1,3) | (2,4) |
| *col_idx* | 0 | 1 | 0 | 2 | 2 | 3 |
| *row_ptr* | 0 | 1 | 2 | 4 | 6 | |

# 【提案手法2】BCCSR 形式

- 実部と虚部を行ごとに交互に配置して，実行列として CSR 形式で格納
- i 行目の実部への参照は row_ptr[2*i]，虚部への参照は row_ptr[2*i+1]
- 実数または純虚数に含まれるゼロを削減
- fp64 と int32 のもとでは，おおざっぱに $m' < 5m/3$ ならばメモリ削減



# 提案手法の適用方法と適用範囲

1. 係数行列 $A$（CSR）を ILU 分解 → 前処理行列 $L, U$（CSR）を得る
2. 前処理行列 $L, U$ を BCCSR 形式に変換
3. 係数行列 $A$ を BCCSR 形式に変換（任意）
   - メモリ使用量によってユーザが使い分け可能
4. 前処理行列 $L, U$ に BC 棄却を適用

CSR → BCCSR 形式への変換は容易

- ILUBiCG 法を含む前処理付きクリロフ部分空間法全般に適用可能
- 「BC 戦略」は CSR 以外の格納形式にも適用可能（と考えている）

① 研究概要と既存手法
② 提案手法（BC 棄却，BCCSR 形式）
# ③ 性能評価
④ まとめ

# 実験に用いる行列

- 行列の多くは SuiteSparse から
- さまざまな応用に由来
- $A\mathbf{x} = \mathbf{b}$ における $A$ として使用

特徴Ⅰがよく表れている
（実部・虚部で非ゼロの偏り）

| 略称 | $n$ | $m$ | $m_{\mathrm{re}}$ | $m_{\mathrm{im}}$ | 対称性 | 背景 |
|---|---|---|---|---|---|---|
| DW | 961 | 10,591 | 10,591 | 6,937 | o | 電磁場解析 |
| QCL | 2,534 | 463,360 | 463,360 | 463,360 | o | 電磁場解析 |
| MPL | 5,962 | 142,190 | 142,190 | 134,954 | o | 振動音響解析 |
| WVG | 21,036 | 303,468 | 303,468 | 11,670 | - | 電磁場解析 |
| WSC | 22,692 | 1,482,390 | 1,482,390 | 1,482,389 | o | モデル縮約 |
| ASH | 23,412 | 218,484 | 218,484 | 19,216 | - | モデル縮約 |
| LIT | 29,282 | 406,084 | 406,084 | 406,084 | - | 電磁場解析 |
| CV1 | 37,365 | 330,633 | 330,633 | 808 | - | 地震波動解析 |
| CV2 | 90,249 | 803,173 | 803,173 | 1,296 | - | 地震波動解析 |
| VFM | 93,476 | 1,434,636 | 1,434,636 | 12,735 | - | 電磁場解析 |
| TMS | 160,013 | 2,527,343 | 2,527,343 | 13,828 | o | 電磁場解析 |
| MON | 169,410 | 5,033,796 | 5,033,796 | 4,887,222 | - | 音響解析 |
| ICJ | 274,087 | 4,137,369 | 3,454,977 | 844,535 | - | 計算化学 |
| CV3 | 381,381 | 3,413,113 | 3,413,113 | 2,760 | - | 地震波動解析 |
| DF3 | 420,408 | 32,886,208 | 32,886,208 | 3,706,513 | o | 電磁場解析 |
| TML | 439,176 | 7,036,670 | 7,036,670 | 29,411 | o | 電磁場解析 |
| DF2 | 607,232 | 25,309,272 | 25,309,272 | 728,900 | o | 電磁場解析 |
| CV4 | 711,450 | 6,376,412 | 6,376,412 | 3,806 | - | 地震波動解析 |

# 実験の仕様

- AVX512 対応の Intel CPU
  - ベクトル最適化はコンパイラ任せ (-march=native)
- ILU(0)分解（ILU分解の一種）を採用
- ABMC 法という既存の高速化手法を適用する場合も含めて検証

  [岩下武史, 高橋康人, 中島浩. 代数ブロック化多色順序付け法による並列化 ICCG ソルバの性能評価. 2009]
  - SpTRSV の並列化のために $A$ の非ゼロ要素の位置を並べ替える手法
  - よく用いられる高速化手法との相性を確かめる目的

| 項目 | 仕様 |
|---|---|
| CPU | Intel Xeon Silver 4314 |
| コンパイラ | GCC 11.4.0 |
| コンパイルオプション | -O3 -march=native |
| OS | Ubuntu 22.04.5 LTS |
| 使用スレッド数 | 32 |

# 実験の流れ

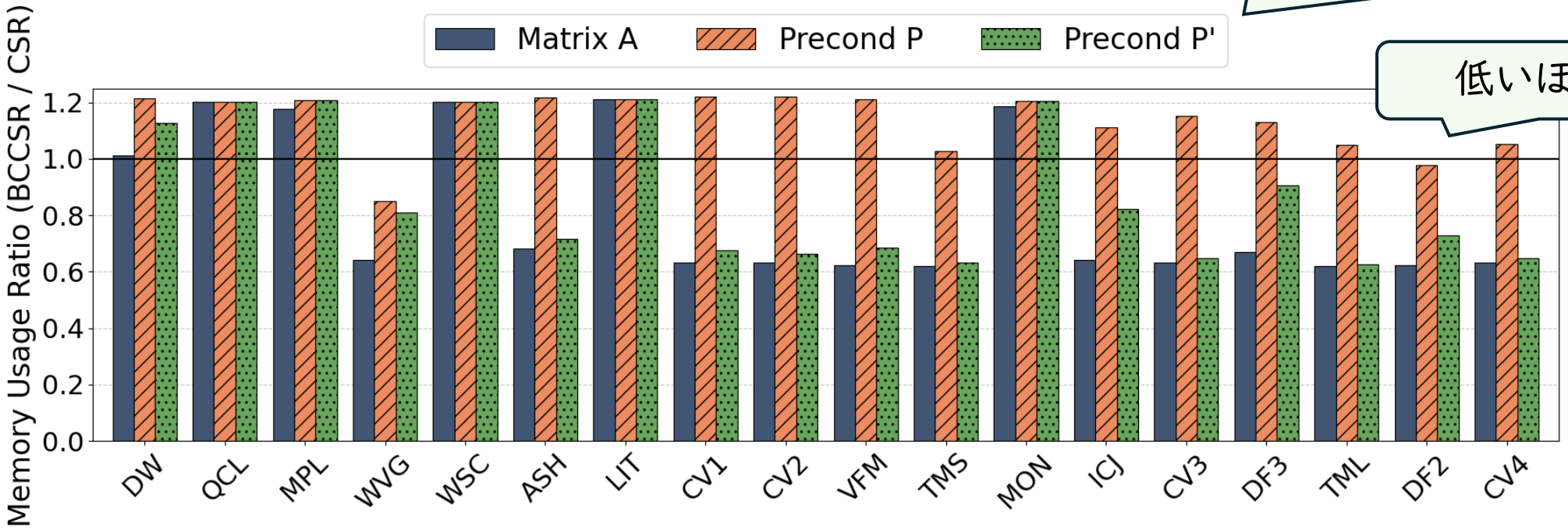
1. BCCSR 形式の性能を CSR 形式と比較する
   - メモリ使用量
   - SpMV・SpTRSV の実行時間
2. BC 棄却（+ BCCSR 形式）の性能を従来の棄却と比較する
   - 収束までに要した反復回数
   - 反復計算の実行時間

# BCCSR 形式によるメモリ削減

- 各行列でメモリ使用量の比 (BCCSR/CSR) を計算
- $A$: 12 個の行列でメモリ使用量を削減．最大 38% 削減
- $P(=L+U)$: ABMC なしではメモリ使用量が増えたが，ABMC ありでは削減された
  - ILU(0) 分解は実部と虚部の両方がゼロでない要素を増やしてしまうが，ABMC 法はこの増加を抑える効果がある
  - ABMC 法と BCCSR 形式の相性 ◎

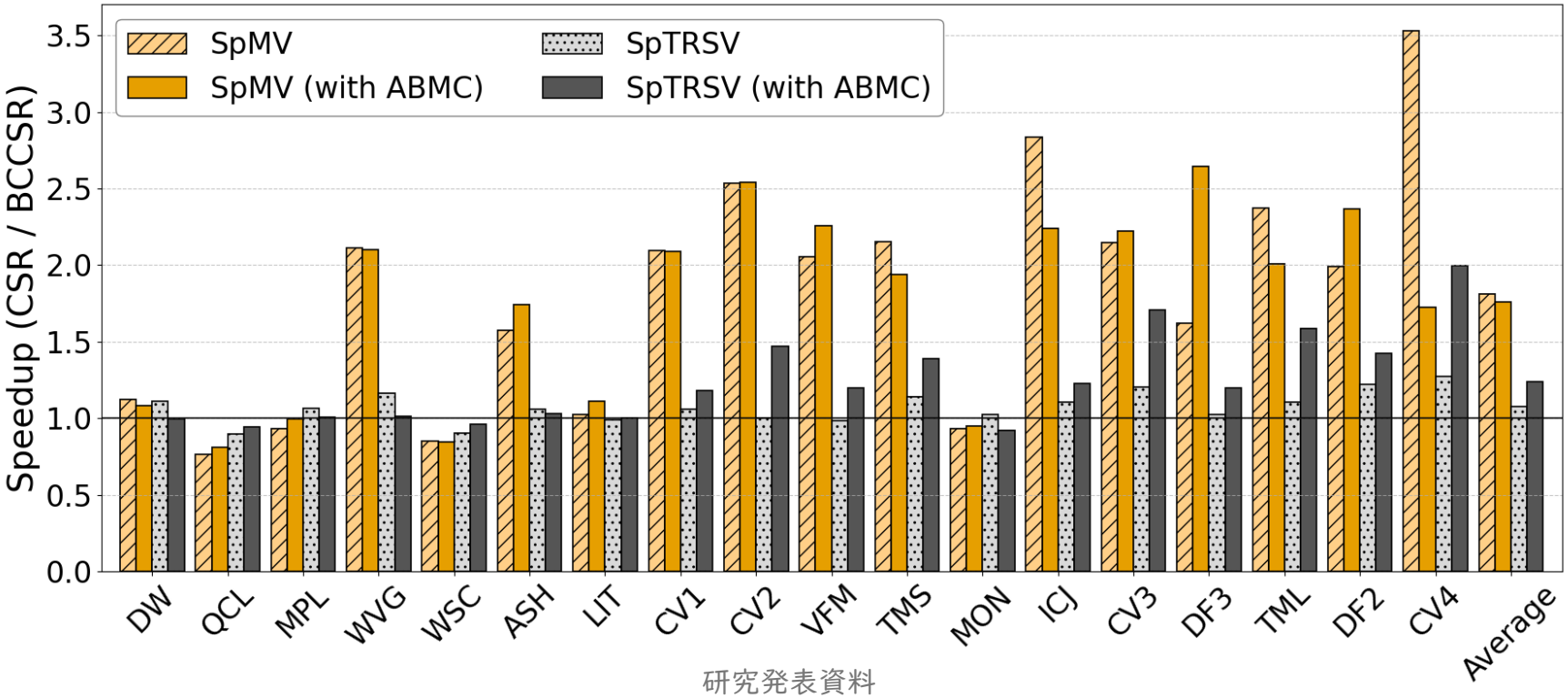
$P'$ = ABMC 法適用時の $P$

低いほど良い



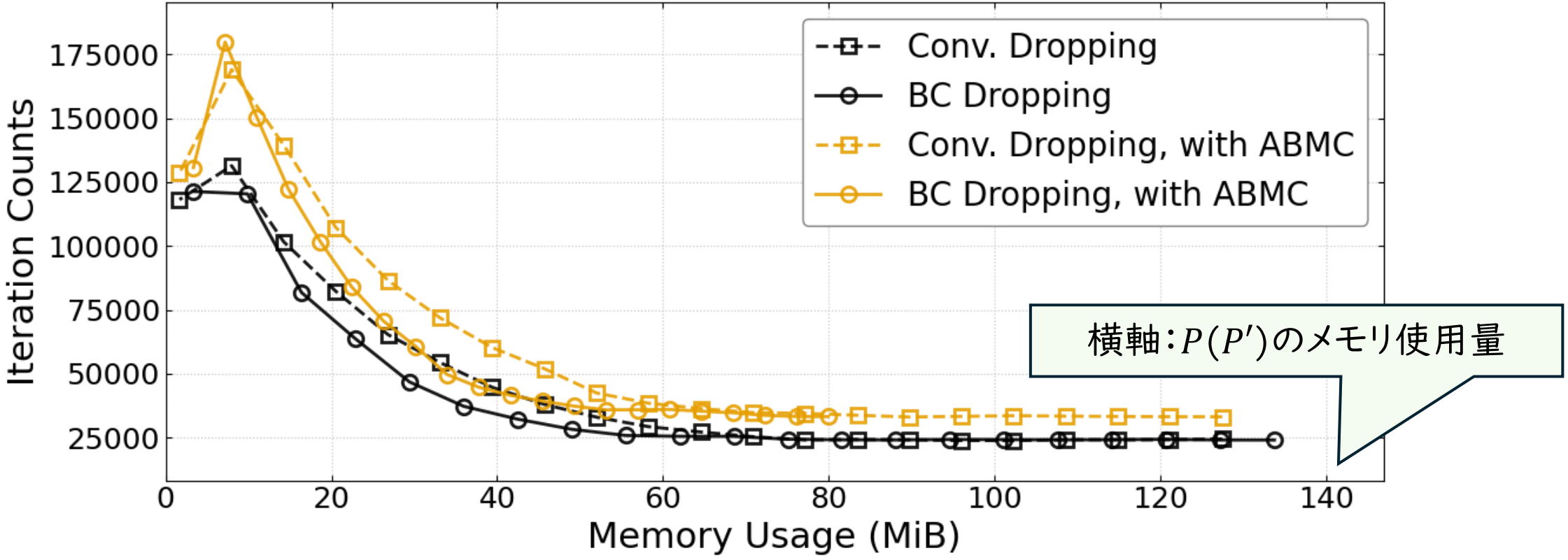
# BCCSR 形式による高速化（SpMV, SpTRSV）

- SpMV, SpTRSV を 1000 回実行し，1回あたりの実行時間を計測
- SpMV では平均 1.8x，最大 3.5x
- SpTRSV (ABMCなし) では平均 1.1x，最大 1.28x
- SpTRSV (ABMCあり) では平均 1.2x，最大 2.00x
  - $P$ のメモリ使用量による差．実行速度面でも ABMC 法との相性 ◎



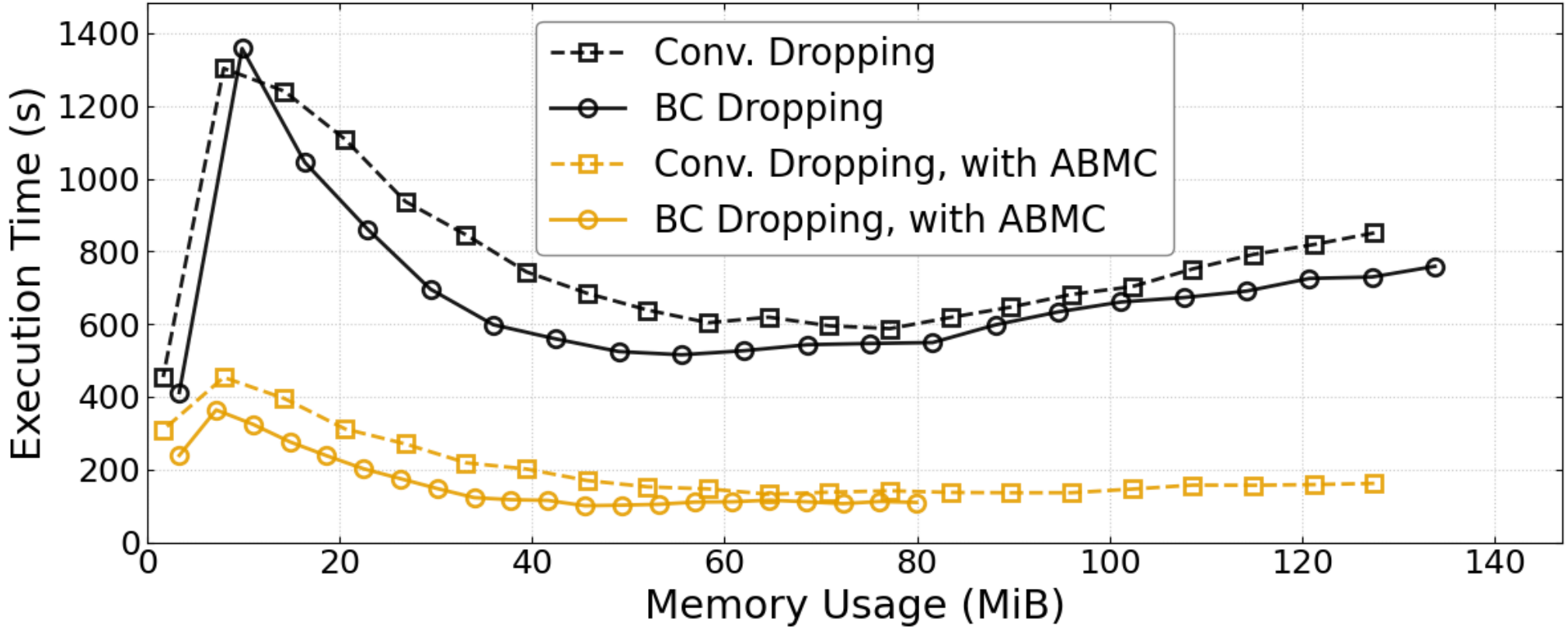
# BC 棄却の影響（反復回数）

- 棄却する要素の割合を変えつつ，反復法を実行（行列 TML）
  - 棄却する割合の下限：何も棄却しない（図における右側）
  - 棄却する割合の上限：対角要素以外全て棄却（図における左側）
- 同じメモリ量のもとでは，全体的に BC 棄却の方が反復回数が小さい😀



# BC 棄却（+BCCSR 形式）の影響（実行時間）

- 棄却する要素の割合を変えつつ，反復法を実行（行列 TML）
  - 反復部の実行時間を計測．$A$ も BCCSR 形式で格納
- 同じメモリ量のもとでは，BC 棄却の方が実行時間が小さい😀
  - BCCSR 形式による性能向上 ＋ BC棄却が反復回数の増加を抑制



① 研究概要と既存手法
② 提案手法（BC 棄却, BCCSR 形式）
③ 性能評価
# ④ まとめ

# 総括

## 目的

- 反復法におけるメモリ使用量，計算時間の削減
- ポストフィルタリング手法における収束性の悪化を抑制

## 手法

- 行列の実部と虚部を分けて格納（BCCSR 形式）・棄却（BC 棄却）

## 結果

- SpMV 最大 3.5x，SpTRSV 最大 2.0x，$A$ のメモリを最大 38% 削減 (CSR比)
- 様々な問題で BC 棄却が収束性悪化を抑制（従来の棄却比）
- ABMC 法と BCCSR 形式の相性◎（$P$ のメモリ削減，SpTRSV の実行速度）

## 今後の課題

- 他の格納形式への「BC」の適用
- BC 棄却についての詳細な解析

# 関連研究（複素行列の分割）

- $A = A_{\mathrm{re}} + iA_{\mathrm{im}}, \mathbf{b} = \mathbf{b}_{\mathrm{re}} + i\mathbf{b}_{\mathrm{im}}$ と表されるとき，以下の実方程式は $A\mathbf{x} = \mathbf{b}$ と同値

$$\begin{pmatrix} A_{\mathrm{re}} & A_{\mathrm{im}} \\ A_{\mathrm{im}} & A_{\mathrm{re}} \end{pmatrix} \begin{pmatrix} \mathbf{x}_{\mathrm{re}} \\ \mathbf{x}_{\mathrm{im}} \end{pmatrix} = \begin{pmatrix} \mathbf{b}_{\mathrm{re}} \\ \mathbf{b}_{\mathrm{im}} \end{pmatrix}$$

- real equivalent form や K-formulation と呼ばれ，研究されてきた
  [Z. G. Huang. Efficient block splitting iteration methods for solving a class of complex symmetric linear systems. 2021.]
  - 複素数演算→実数演算で演算効率とメモリ使用量改善 😀
  - 収束性が低下する場合もある 😖
- $A = H + S$（$H$：エルミート，$S$：歪エルミート）に分割して，以下で反復する HSS 法
  [Z. Z. Bai, G. H. Golub, and M. K. Ng. Hermitian and Skew-Hermitian Splitting Methods for Non-Hermitian Positive Definite Linear Systems. 2003.]

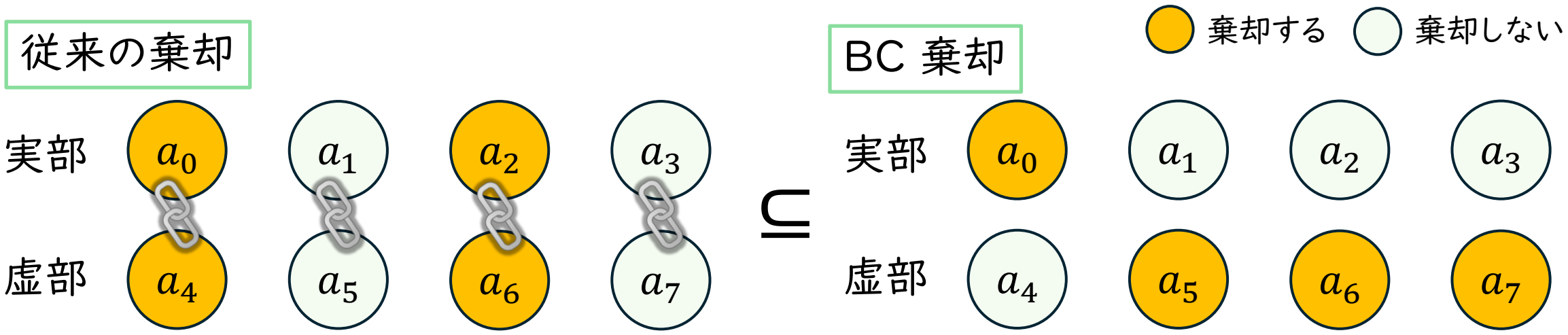
$$(\alpha I + H)x_{k+\frac{1}{2}} = (\alpha I - S)x_k + b$$

$$(\alpha I + S)x_{k+1} = (\alpha I - H)x_{k+\frac{1}{2}} + b$$

  - 発展として，内部反復に CG 法（クリロフ部分空間法の一つ）を用いる IHSS 法も提案
- クリロフ部分空間法は依然として有効な方針

# 定理の証明

- 非ゼロ要素における実部および虚部の二乗の値を並べた配列 $\mathbf{a}$
- $\mathbf{a}$ から要素を $k$ 個選んだ総和 $=$ $k$ 個の実数要素を棄却したときの変更量の総和
- BC 棄却の方が, 選び方の制約が真に緩い
  - 従来の棄却: $(a_0, a_m), (a_1, a_{1+m}), \ldots, (a_{m-1}, a_{2m-1})$ から $k/2$ 個のペアを選ぶ
  - BC 棄却: $a_0, \ldots, a_{2m}$ から自由に $k$ 個選ぶ
- 両手法は, それぞれの制約のもとで総和が最小になる選び方をする □



# BCCSR 形式での SpMV カーネル

- CSR 形式と同様, 行ごとに計算
- 実部と虚部の寄与を足せばよい

```
for (int i = 0; i < n; ++i) {
    sum = 0;
    for (int k = row_ptr[i]; k < row_ptr[i+1]; ++k) {
        sum += val[k] * x[col_idx[k]];
    }
    result[i] = sum;
}
```

```
for (int i = 0; i < n; ++i) {
    sum = 0;
    for (int k = row_ptr[2*i+0]; k < row_ptr[2*i+1]; ++k) {
        sum += val[k] * x[col_idx[k]];
    }
    for (int k = row_ptr[2*i+1]; k < row_ptr[2*i+2]; ++k) {
        sum += val[k] * x[col_idx[k]] * 1.0i;
    }
    result[i] = sum;
}
```

# BCCSR 形式での SpTRSV カーネル

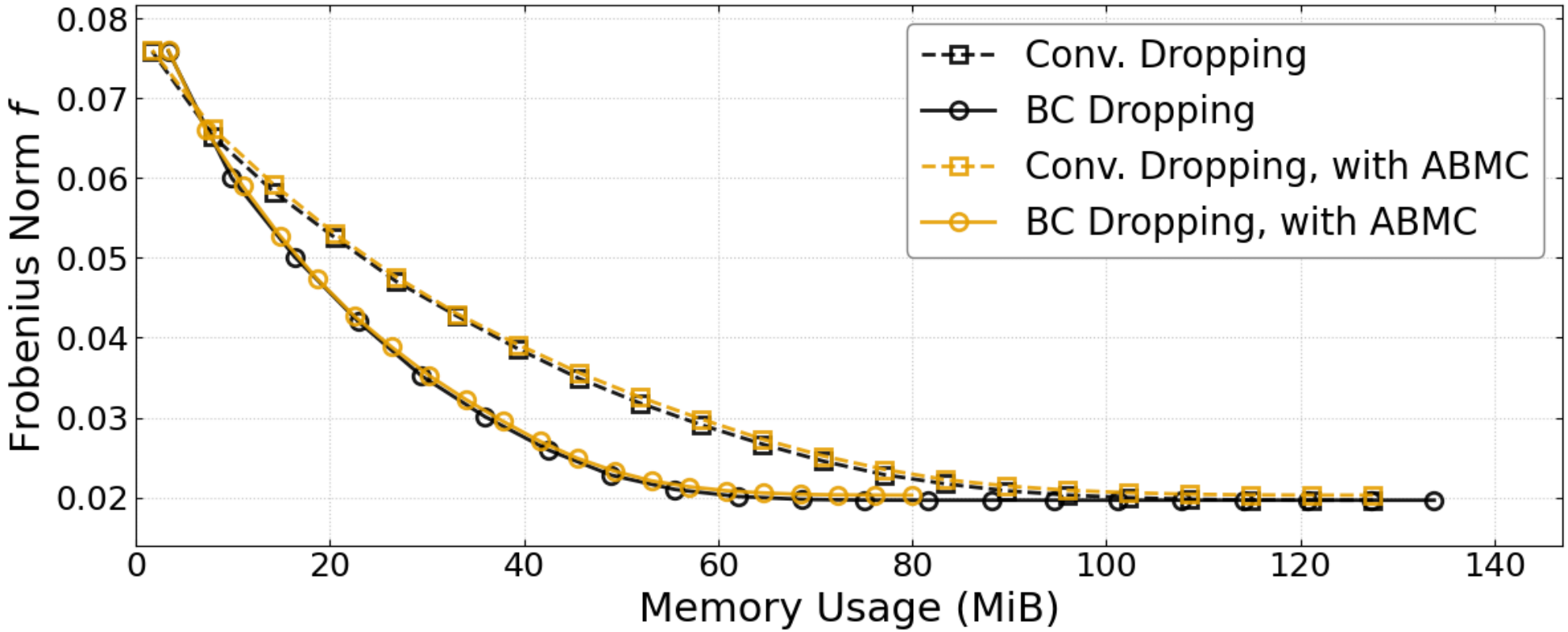
- CSR 形式と同様, 行ごとに計算
- 実部と虚部の寄与を引けばよい

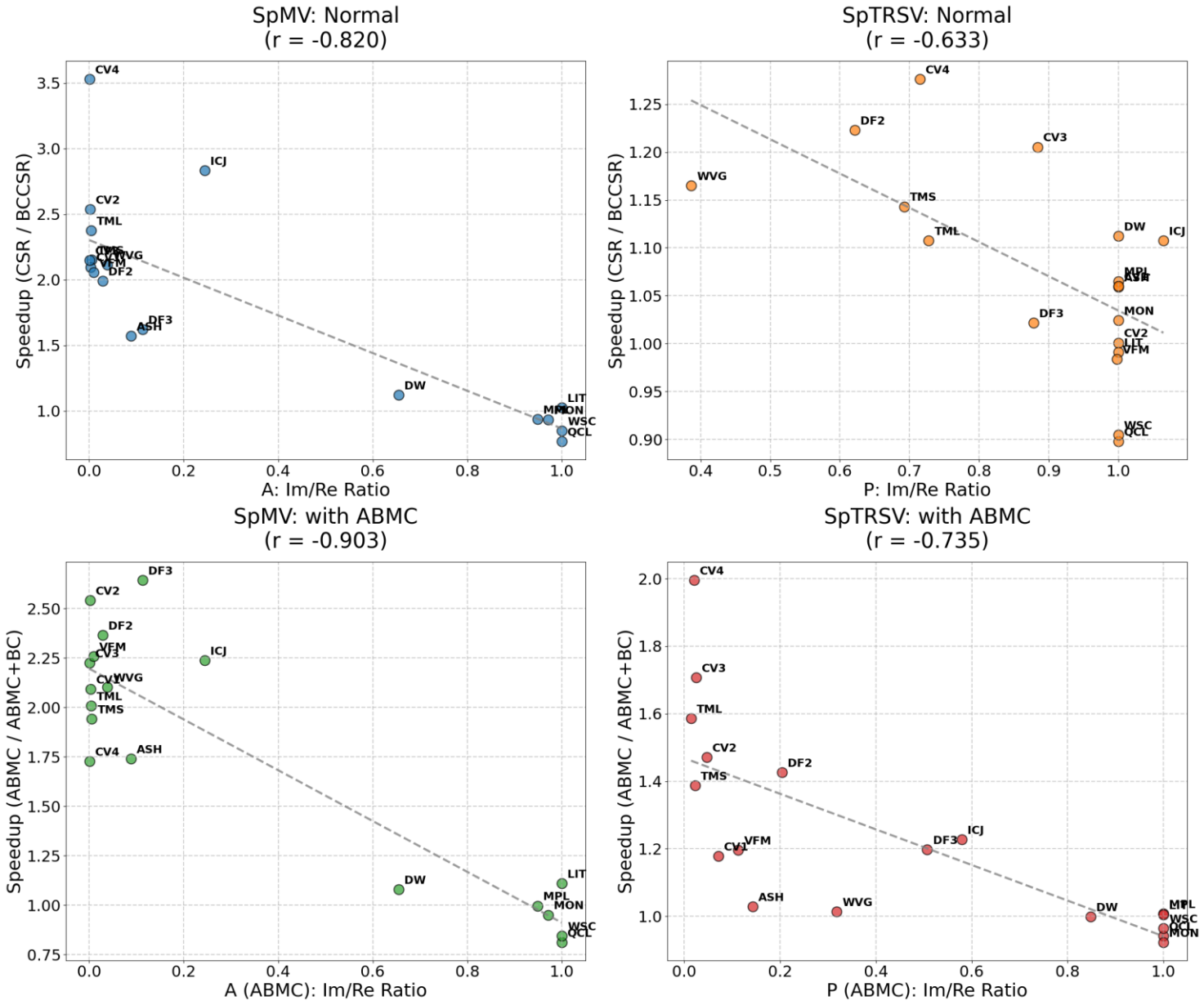
```
for (int i = 0; i < n; ++i) {
    sum = 0;
    for (int k = row_ptr[i]; k < row_ptr[i+1]; ++k) {
        sum += val[k] * x[col_idx[k]];
    }
    x[i] = b[i] - sum;
}
```

```
for (int i = 0; i < n; ++i) {
    sum = 0;
    for (int k = row_ptr[2*i+0]; k < row_ptr[2*i+1]; ++k) {
        sum += val[k] * x[col_idx[k]];
    }
    for (int k = row_ptr[2*i+1]; k < row_ptr[2*i+2]; ++k) {
        sum += val[k] * x[col_idx[k]] * 1.0i;
    }
    x[i] = b[i] - sum;
}
```

# BC 棄却の影響（指標値 $f$）

- 棄却する要素の割合を変えつつ，反復法を実行（行列 TML）
  - 指標値 $f := ||R||_F = \sqrt{\Sigma_i \Sigma_j (R_{ij})^2}$ を計測．低いほどよい近似
- 同じメモリ量のもとでは，全体的に BC 棄却の方が指標値 $f$ が小さい😀



# 実/虚部の非ゼロ要素数の偏りと高速化率の関係

偏りのある行列ほど
高速化